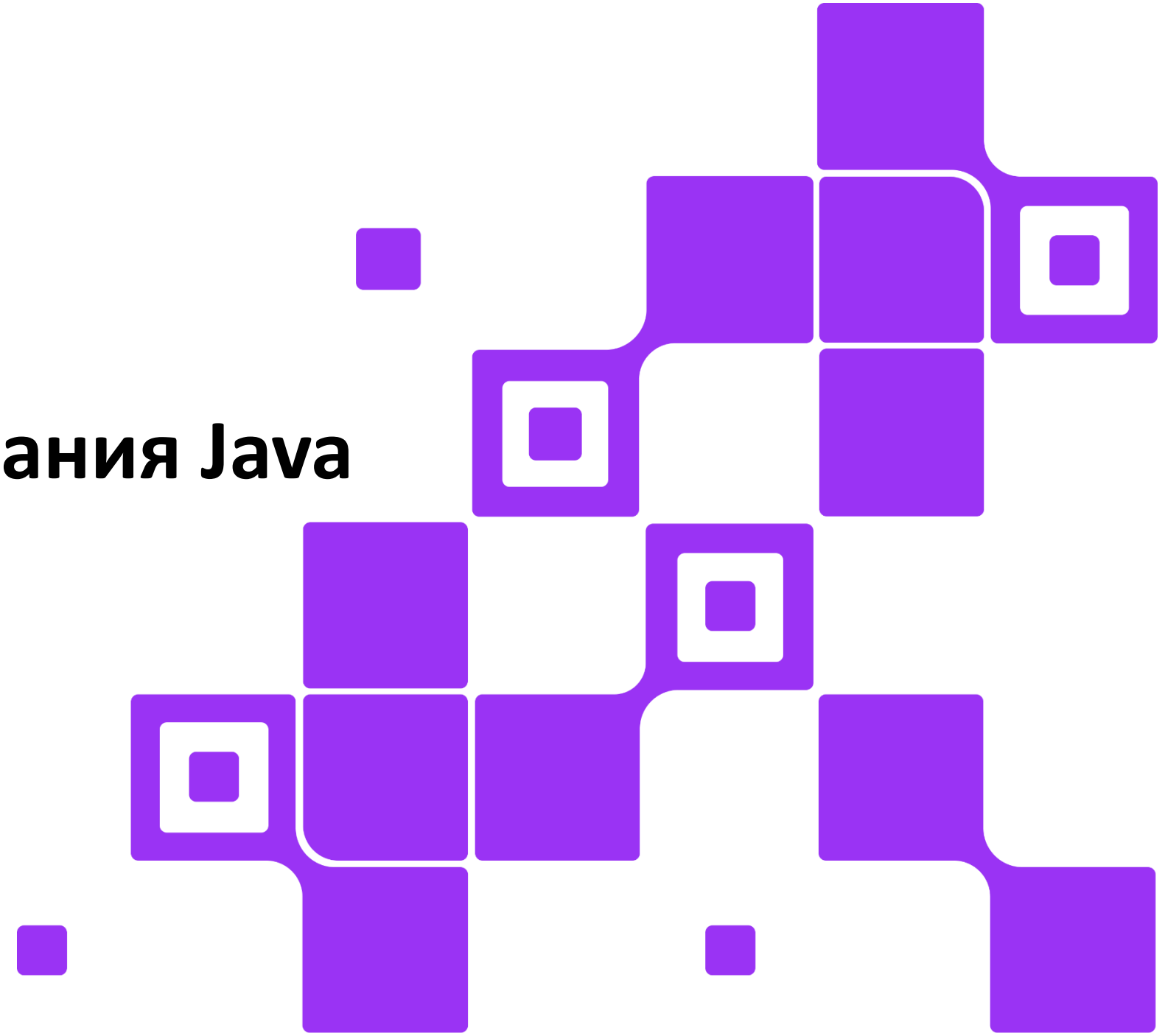
Язык программирования Java



IThub

Давайте познакомимся

Захарова Александра Юрьевна

- НГТУ им. Алексеева магистр «Информационные системы и технологии».
- С 2019 по 2025 инженер-программист Java компания Netcracker (путь от Junior до Senior).
- С 2025 ассистент кафедры «Информатика и системы управления» (НГТУ им. Алексеева), дисциплина «Программирование кроссплатформенных приложений».



Java - строго типизированный объектно-ориентированный язык программирования общего назначения и вычислительная платформа, созданные компанией Sun Microsystems в 1995 году (впоследствии приобретенной Oracle).

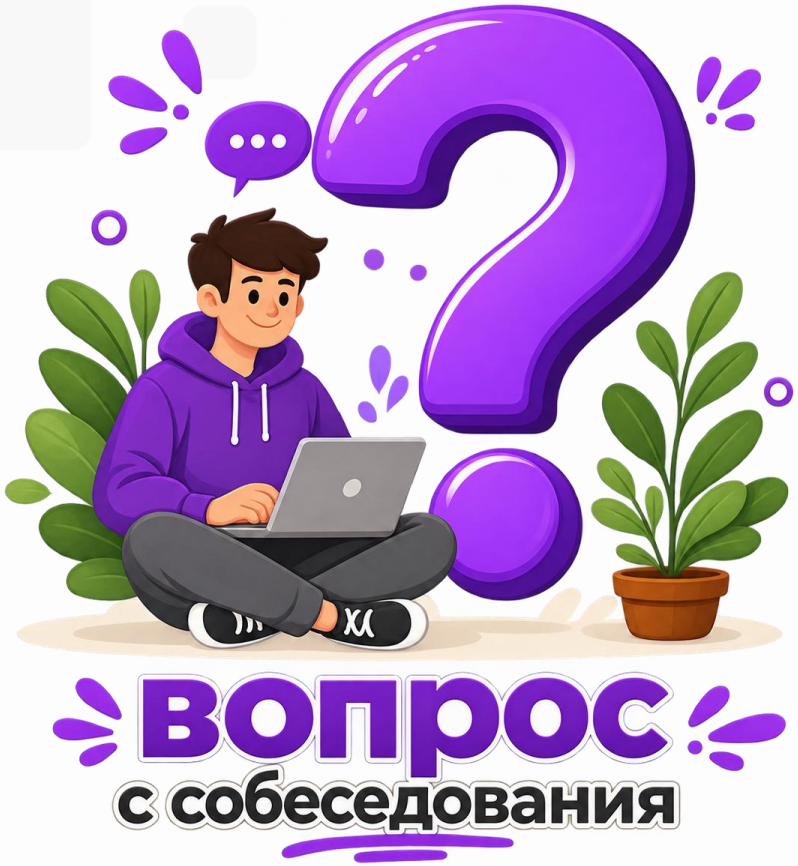
- интернет-магазины
- банковские системы
- телеком-операторы
- Android-приложения (Kotlin)

JRE

JRE (Java Runtime Environment) — это среда выполнения Java, набор программного обеспечения, необходимый для запуска приложений, написанных на языке программирования Java. Она включает в себя виртуальную машину JVM (Java Virtual Machine) и набор библиотек классов, обеспечивая работу Java-программ на компьютере, но не используется для их разработки.

- **Назначение**: JRE необходима пользователям для запуска игр, веб-сайтов и программ, созданных на Java.
- **Состав**: Основной компонент — JVM (Java Virtual Machine), которая выполняет байт-код Java, а также стандартные библиотеки классов (набор готовых инструментов).
- **Отличие от JDK**: JRE предназначена для выполнения программ, в то время как JDK (Java Development Kit) используется разработчиками для создания и компиляции кода.
- **Платформозависимость**: JRE подбирается под конкретную операционную систему (Windows, Linux, macOS).

IThub



За что отвечает JVM?

Виртуальная машина Java (Java Virtual Machine) - это механизм, предоставляющий среду выполнения для управления Java-кодом или приложениями. Виртуальная машина является независимой оболочкой исполнения кода, благодаря которой возможен её запуск на любой ОС, без влияния ОС на выполняемую программу.

- Загрузка, проверка и исполнение байт кода.
- Предоставление среды выполнения для выполнения байт-кода.
- Управление памятью и очисткой мусора (Garbage collection).

Компиляция программы

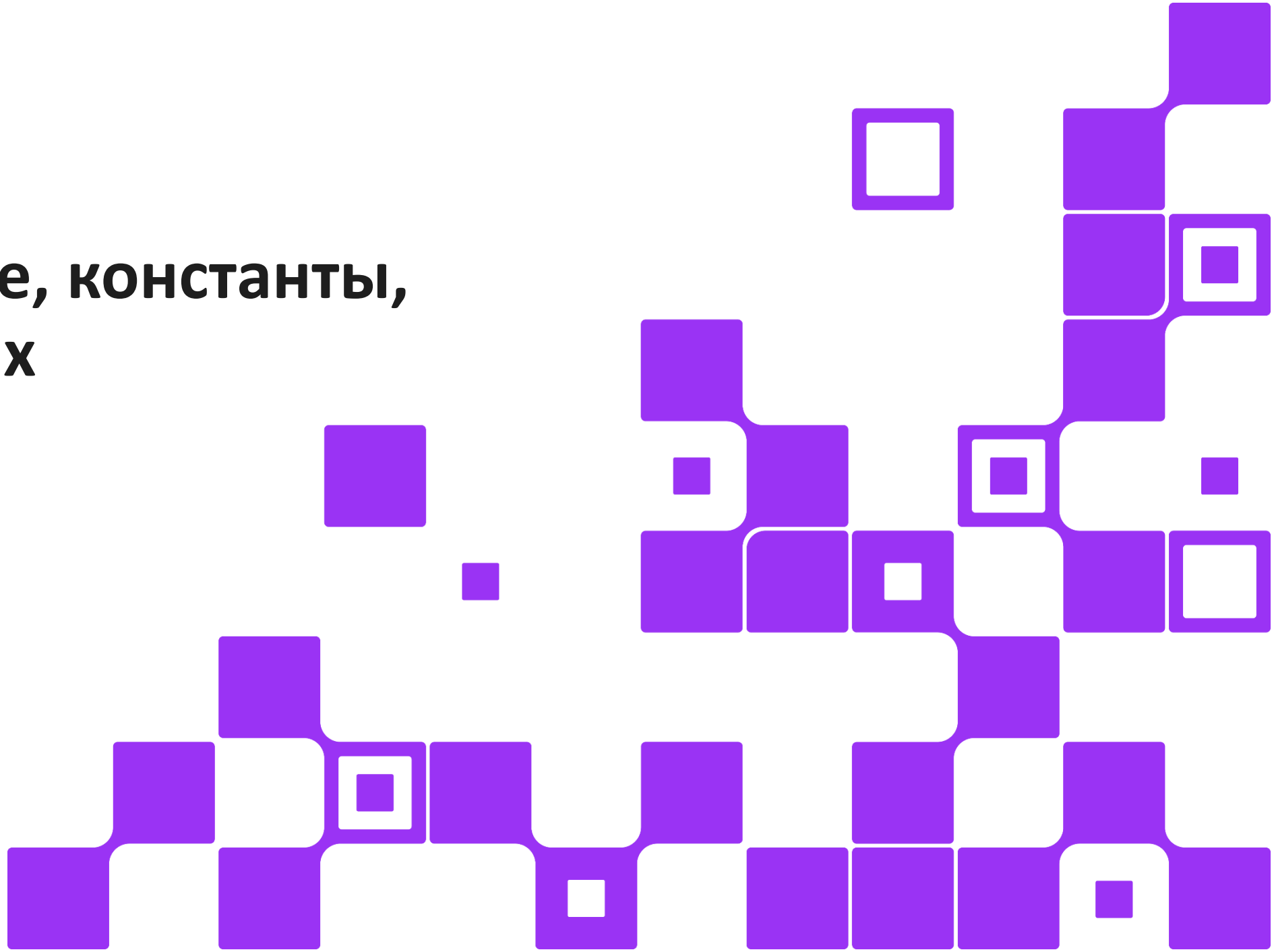
```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

```
E:\Programming\javaexamples\src\main\java\org\example>javac Main.java
E:\Programming\javaexamples\src\main\java\org\example>cd
E:\Programming\javaexamples\src\main\java
E:\Programming\javaexamples\src\main\java>java org.example.Main
Hello World!
```

1. Компилятор javac – в командной строке имя файла с исходным кодом
2. Компилятор создает файл .class с байт-кодом
3. Загрузчик приложений java (JVM)

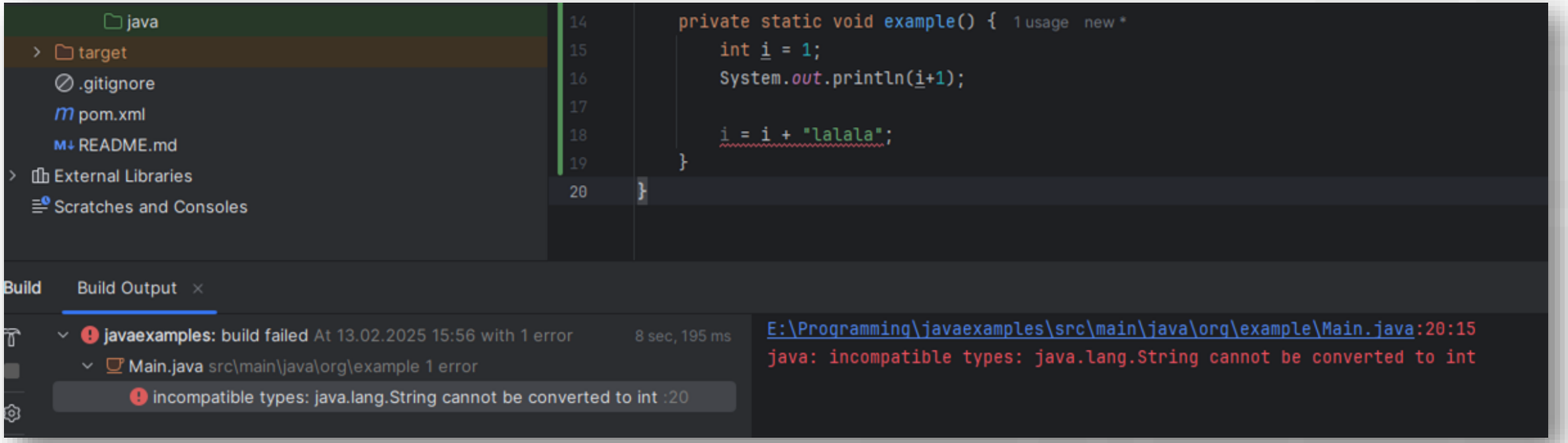
IThub

Переменные, константы, типы данных



Java - строго типизированный язык

Компилятор Java проверяет все выражения и параметры для гарантирования совместимости типов. Любые несоответствия типов считаются ошибками, которые должны быть исправлены до того, как компилятор завершит компиляцию класса.



IThub

Для хранения данных в программе предназначены **переменные**. Переменная представляет именованную область памяти, которая хранит значение определенного типа. Каждая переменная имеет **тип, имя (идентификатор) и значение**. Тип определяет, какую информацию может хранить переменная или диапазон допустимых значений.

Примитивные типы

В Java определены восемь примитивных типов данных:

- ***byte***
  - 8-битное целое
  - от -128 до 127
- ***short***
  - 16-битное целое
  - от -32 768 до 32 767
- ***int***
  - 32-битное целое
  - От -2 147 483 648
    до 2 147 483 647
- ***long***
  - 64-битное целое
  - От -9 223 372 036 854 775 808
    до 9 223 372 036 854 775 807
- ***char***
  - 16-битный символ Unicode
  - от 0 до 65535
- ***float*** (в качестве разделителя целой и дробной части используется точка)
  - 32-битное число с плавающей точкой
- ***double***
  - 64-битное число с плавающей точкой
- ***boolean***
  - логическое значение
  - ***true*** / ***false***

Идентификаторы

IThub

Применяются для именования классов, переменных и методов.

Не должны начинаться с цифры, чтобы компилятор не путал их с числовым литералом

Java чувствителен к регистру, поэтому VALUE и Value - разные идентификаторы

Идентификаторы

IThub

Имена должны передавать намерения программиста

```
int d; // Прошедшее время

int daysSinceCreation;
```

Избегайте дезинформации

Используйте удобопроизносимые имена

```
class Customer {                    class Customer {
private Date genymdhms;             private Date generationTimestamp;
```

Выбирайте имена, удобные для поиска

```
int e                               int MAX_CLASSES_PER_STUDENT
```

! однобуквенные имена могут использоваться ТОЛЬКО для локальных переменных в коротких методах



Идентификаторы

IThub

Избегайте схем кодирования имен

Не добавляйте избыточный контекст

Имена классов и объектов должны представлять собой существительные и их комбинации: Customer, Account и AddressParser. Имя класса не должно быть глаголом

Имена методов представляют собой глаголы или глагольные словосочетания: postPayment, deletePage, save и т. д. Методы чтения/записи и предикаты образуются из значения и префикса get, set и тп.



Давайте объявим переменную и выведем её в консоль.

1. Откроем Intellij idea.
2. Создадим новый maven-проект.
3. Создадим папку java.
4. Создадим класс Main и метод main().
5. В классе Main в методе main() напишем простой пример.

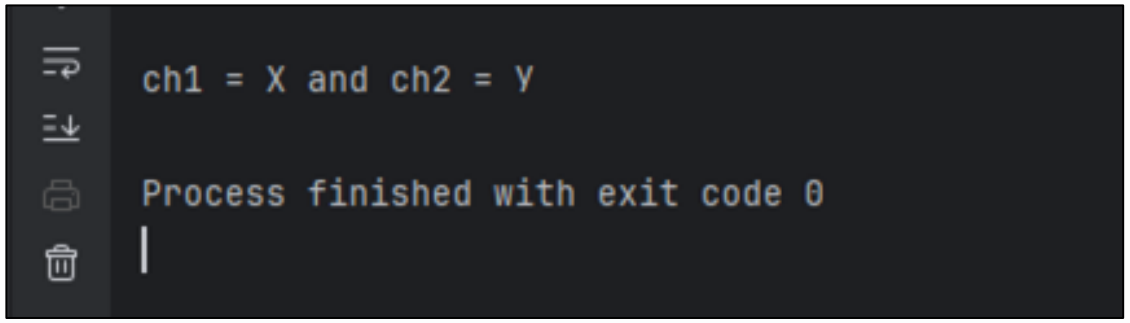
```
public class Main {

    public static void main(String[] args) {
        int i = 1;
        System.out.println(i);
    }
}
```

Символы (char)

IThub

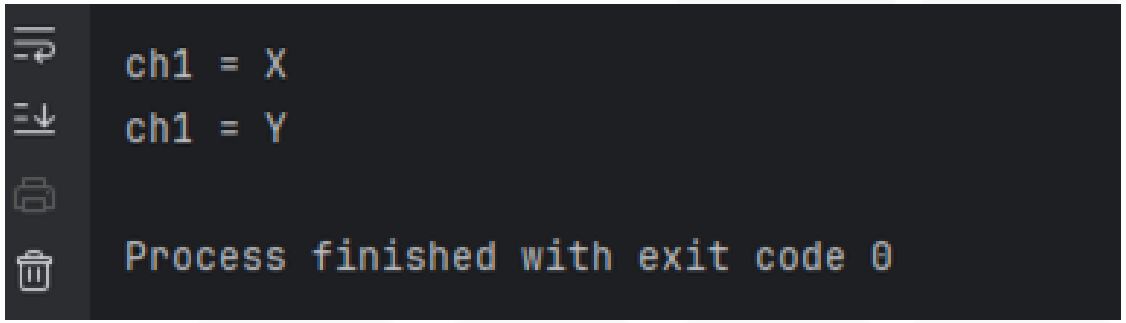
```
public void charDemo() {
    char ch1 , ch2 ;
    ch1 = 88; // code for x
    ch2 = 'У';
    System.out.println("ch1 = " + ch1 + " and ch2 = " + ch2);
}
```



Переменной ch1 присвоено значение 88, которое является кодом ASCII (и Unicode), соответствующим букве X

```
public void charDemo2() {
    char ch1 = 'X';
    System.out.println("ch1 = " + ch1);

    ch1++;
    System.out.println("ch1 = " + ch1);
}
```



Переменной ch1 сначала присваивается значение ‘X’. Затем выполняется инкрементирование ch1. В результате ch1 содержит значение 'У', т.е. следующий символ в последовательности ASCII (и Unicode).

ПРЕОБРАЗОВАНИЕ И ПРИВЕДЕНИЕ ТИПОВ

Автоматическое преобразование типов происходит в случае удовлетворения следующих двух условий:

- два типа совместимы
- целевой тип больше исходного типа

Например, тип int всегда достаточно велик, чтобы хранить все допустимые значения byte, поэтому явное приведение не требуется

- ***Byte*** от -128 до 127 (диапазон = 256)
- ***Int*** От -2 147 483 648 до 2 147 483 647

```
public void conversionDemo() {
    byte b;
    int i = 257;
    double d = 323.142;

    System.out.println( " \n int to byte" );
    b = (byte) i;
    System.out.println( "i and b : " + i + " " + b);

    System.out.println( " \n double to int" );
    i = (int) d;
    System.out.println( "d and i : " + d + " " + i );

    System.out.println ( " \n double to byte" );
    b = (byte) d;
    System.out.println ( "d and b : " + d + " " + b);
}
```

- Если целочисленное значение выходит за пределы диапазона типа byte, тогда оно уменьшается по модулю (остатку от целочисленного деления) диапазона byte.

  остаток от деления 257 на 256 равен 1.

- Когда значение переменной d преобразуется в тип int, его дробная часть утрачивается
- Когда значение переменной d преобразуется в тип byte, его дробная часть теряется, а значение уменьшается по модулю 256, что в этом случае дает 67

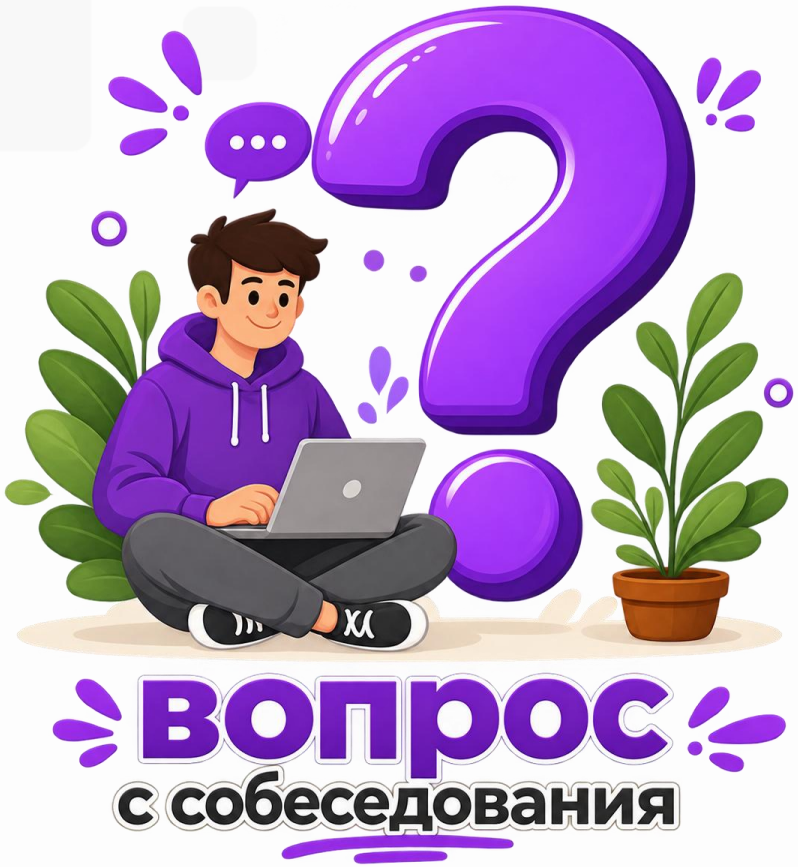
```
 int to byte
i and b : 257 1

 double to int
d and i : 323.142 323

 double to byte
d and b : 323.142 67

Process finished with exit code 0
```

IThub



Что такое понижение и повышение типа?

Расширение (повышение, **upcasting**) примитивного типа означает, что осуществляется переход от менее емкого типа к более ёмкому. Например, от типа byte (длина 1 байт) к типу int (длина 4 байта). Такие преобразование безопасны в том смысле, что новый тип всегда гарантировано вмещает в себя все данные, которые хранились в старом.

Сужение (понижение, **downcasting**) примитивного типа означает, что переход осуществляется от более емкого типа к менее емкому. При таком преобразовании есть риск потерять данные.

Консольный ввод

Для получения ввода с консоли необходимо использовать класс (тип) **Scanner.**

Давайте попробуем реализовать простую программу, где пользователь будет вводить свой возраст, а программа будет выводить его нам в ответ.

```
public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int age = scanner.nextInt();
        System.out.println("Вам: " + age + " лет");
    }
}
```

Методы класса Scanner

IThub

scan.nextInt()	Мы запрашиваем int из консоли
scan.nextBoolean()	Мы запрашиваем boolean из консоли
scan.nextByte()	Мы запрашиваем byte из консоли
scan.nextLong()	Мы запрашиваем long из консоли
scan.nextShort()	Мы запрашиваем short из консоли
scan.nextFloat()	Мы запрашиваем float из консоли

Во время того, как программа запрашивает что либо из консоли, пока пользователь не совершит ввод, программа не будет продолжать исполнение кода.

Арифметические операции

Операция	Описание
+	сложение
-	вычитание
*	умножение
/	деление
%	остаток от деления (деление по модулю)
++	операция инкремента (увеличение на 1)
--	операция декремента (уменьшение на 1)
+=	сложение с присваиванием
-=	вычитание с присваиванием
*=	умножение с присваиванием
/=	деление с присваиванием
%=	деление по модулю с присваиванием

Разработка через тестирование (TTD)

Основные идеи TDD (Test-Driven Development):

- создание автоматических тестов до написания функционального кода
- постоянная проверка кода на соответствие требованиям
- обеспечение возможности быстрого обнаружения и устранения дефектов

Первый закон. Не пишите код продукта, пока не напишете отказной модульный тест.

Второй закон. Не пишите модульный тест в объеме большем, чем необходимо для отказа. Невозможность компиляции является отказом.

Третий закон. Не пишите код продукта в объем большем, чем необходимо для прохождения текущего отказного теста.



Задание

Рассмотрим все виды операций, используя метод разработки через тестирование.

Задание

На вход программе необходимо дать число, обозначающее возраст пользователя. На выходе программа должна ответить:

1) Сколько лет пользователю в данный момент?
2) Сколько лет будет пользователю через 5 лет?
3) Сколько лет было пользователю в прошлом году?
4) Сколько лет будет пользователю в следующем году?

Приоритет арифметических операций

Одни операции имеют больший приоритет, чем другие, и поэтому выполняются вначале. Операции в порядке уменьшения приоритета:

1. ++ (постфиксный инкремент)
2. -- (постфиксный декремент)
3. ++ (префиксный инкремент)
4. -- (префиксный декремент)
5. * (умножение)
6. / (деление)
7. % (остаток от деления)
8. + (сложение)
9. - (вычитание)

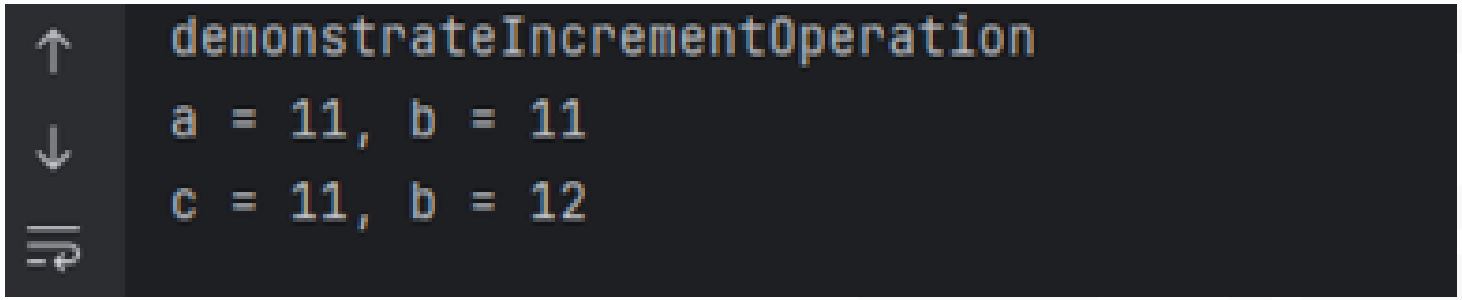
Приоритет операций следует учитывать при выполнении набора арифметических выражений.

IThub

```java
public void demonstrateIncrementOperation(int number) {
    System.out.println("\ndemonstrateIncrementOperation");
    int a = number;

    // префиксный инкремент
    int b = ++a; // вначале значение переменной a увеличивается на 1, а затем ее значение присваивается переменной b
    System.out.println("a = " + a + ", b = " + b);

    // постфиксный инкремент
    int c = b++; // вначале значение переменной c присваивается переменной b, а потом значение переменной b увеличивается на 1
    System.out.println("c = " + c + ", b = " + b);
}
```

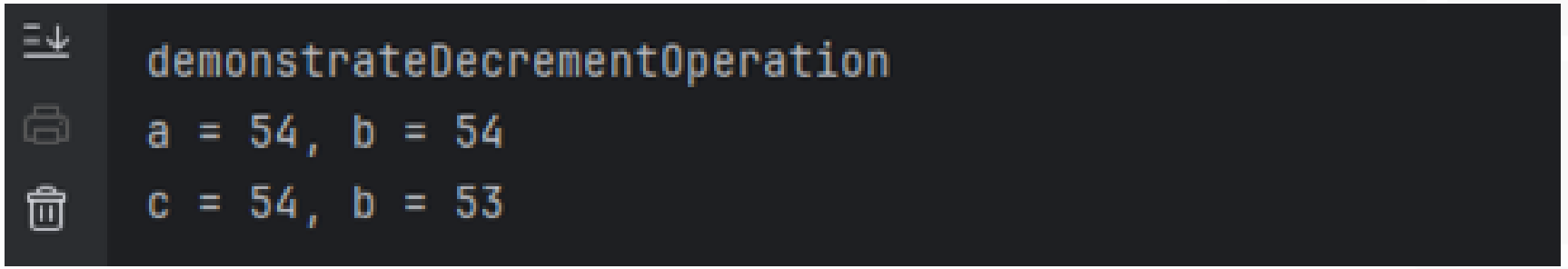


IThub

```java
    public void demonstrateDecrementOperation(int number) {
        System.out.println("\ndemonstrateDecrementOperation");

        int a = number;
        // префиксный декремент
        int b = --a; // вначале значение переменной a уменьшается на 1, а потом ее значение присваивается переменной b
        System.out.println("a = " + a + ", b = " + b);

        // постфиксный инкремент
        int c = b--; // вначале значение переменной c присваивается переменной b, а потом значение переменной b уменьшается на 1
        System.out.println("c = " + c + ", b = " + b);
    }
```



IThub

Комментарии

- одна из причин написания комментариев - низкое качество кода

- не используйте избыточные комментарии

```
// версия
private String version;
```

- объясните намерение в коде – плохой код надо исправлять, а не комментировать

```
// Проверить, положена ли работнику полная премия
if ((employee.flags & HOURLY_FLAG) &&
(employee.age > 65))
```

```
if (employee.isEligibleForFullBenefits())
```

- информативные комментарии

```
// Поиск по формату: kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
"\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

- комментарии ToDo

```
// TODO ...
```

- если вы разрабатываете API для общего пользования, несомненно, для него следует написать хорошие комментарии Javadoc



Форматирование

- форматирование кода важно
- разделяйте разные смысловые блоки пустыми строками
- переменные следует объявлять как можно ближе к месту использования
- переменные экземпляров, напротив, должны объявляться в начале класса
- если одна функция вызывает другую, то эти функции должны располагаться вблизи друг от друга по вертикали
- взаимозависимые функции должны размещаться в нисходящем порядке
- строка должна быть максимально короткой
- группа разработчиков согласует единый стиль

форматирования

